Chapter 24 — Mock Interview Drills (Study Edition)

> **Goal.** Turn each of the 23 classic object-oriented design problems into a timed, spoken rehearsal. A drill is the interview script only: the prompt, the questions to ask back, a plan, a checklist, the usual follow-ups and a link. The designs themselves are taught in the case-study chapters; this chapter trains you to present them.

An **object-oriented design (OOD) round**, also called a low-level design (LLD) round, is a 45–60 minute interview in which you turn a one-line product idea into classes, relationships and a little code while explaining your choices. It grades the **model** and the **explanation**. These drills train the explanation.

24.0 How to use these drills

Running a drill

1. Read only the prompt and the clarifying questions. Cover the rest.
2. Start a 45-minute timer and talk out loud, sketching on paper and typing the core code.
3. When the timer stops, score yourself with the rubric below **before** looking at anything.
4. Uncover the checklist and follow-ups, then open the linked case study. Re-score, and write down the one dimension you will fix next time.
5. Repeat a drill the next day if you scored under 24.

The timebox

Every drill's "Minute-by-minute plan" uses these phases. Only the problem-specific content is listed in each drill.

45-min round	60-min round	Phase	What exists at the end
0–5	0–6	**Clarify**	6–10 numbered requirements, an out-of-scope line, 2–4 actors (people or systems that use the design)
5–8	6–10	**Use cases**	4–8 actions, one line each
8–18	10–22	**Entities and relationships**	Classes with one sentence of responsibility each; a class diagram with kinds and multiplicities (how many on each side)
18–23	22–28	**Walk one flow**	The main use case traced through the classes
23–35	28–45	**Code**	The 2–3 classes that carry the main flow, written properly; the rest only named
35–45	45–60	**Follow-ups**	Concurrency, one extension, trade-offs

Two rules for every problem. **Say the plan in the first minute**, so the interviewer can steer early. **Have a class diagram by minute 18**; if you don't, cut scope out loud ("refunds later") and move on. Extra time in a 60-minute round goes to code and follow-ups, never to longer clarifying.

How to explain clearly

Narrate **decisions, not actions**. "I'm drawing Ticket" is an action. "Ticket is its own class because it outlives the parking session and payment needs something to point at" is a decision, and decisions are what get graded.

- **Each class:** name, one sentence of responsibility, why it is separate.
- **Each relationship:** kind (composition, aggregation, association; §3.2–3.8) and multiplicity.
- **Each design pattern:** name the **change it absorbs**. "Strategy for pricing, because pricing changes most often and a new rule should be a new class" scores; "I'll use Strategy" alone does not.

- **At each phase boundary**, check in: "That's my class list. Shall I walk the booking flow, or would you rather I start on concurrency?"

Terms the drills use

Pattern and concurrency terms (Strategy, Command, compare-and-set, idempotent, half-open interval …) are defined in the glossary, §22.11. In the plans, "`A` owns many `B`" means composition, one to many, and "walk" means trace one use case through the classes.

Scoring rubric

Score each dimension 1–4 (total out of 28).

Dimension	1 — Weak	2 — Partial	3 — Solid	4 — Strong
Requirements	Drew classes at once	Asked questions, wrote nothing	Numbered functional list plus out of scope	Also non-functional (concurrency, failure), confirmed with the interviewer
Entities	One "Manager" does everything	Right nouns, blurred jobs	One-sentence responsibility per class	Also the non-obvious split (type vs instance, book vs copy)
Relationships	Unlabelled lines	Arrows without kind or multiplicity	Correct kinds and multiplicities	Also inheritance only where substitution holds, with reasons
Patterns	None, or named for show	Named, change not stated	One or two, each tied to the change it absorbs	Also one considered and rejected, with the reason
Concurrency	Not mentioned	"Add synchronized" with no target	Names the shared resource; locks at the right grain	Also the failure mode: hold expiry, lock order, idempotent retry
Extensibility	A new type edits many classes	Claims extensibility without showing it	Shows one extension as "one new class"	Also says what would *not* extend cheaply and why that is acceptable
Communication	Silent or rambling	Narrates actions	Narrates decisions, checks in	Also keeps to time and invites steering

Reading the total: treat 24–28 as interview-ready. 18–23 is borderline, usually dragged down by Requirements or Communication. Below 18, study the case and repeat the drill tomorrow.

Suggested order

Small state machines first, platforms last: Parking Lot, Vending Machine, ATM, Elevator, Amazon Locker → Library, Movie Tickets, Hotel, Car Rental, Airline, Restaurant, Meeting Scheduler → Blackjack, Chess, Jigsaw → Amazon Shopping, Stack Overflow, Facebook, LinkedIn, ESPNcricinfo, Stock Brokerage → Ride Sharing and Food Delivery with no notes at all. The sections below follow the original course order so you can find a problem quickly.

24.1 Drill — Parking Lot

The prompt as the interviewer says it

> "Design a parking lot. Cars come in, get a ticket, park, and pay when they leave. Take it wherever you like."

The open ending tests whether you bound the problem.

Clarifying questions to ask

Ask	Assume
Levels?	Several, each with a fixed set of spots
Vehicle and spot types?	Motorbike/car/truck; spots small/medium/large; a smaller vehicle may take a bigger spot
Gates?	Several entries and exits running at once
Fee?	Hourly, rate per vehicle type, easy to change
Display?	Free spots per level and size
Out of scope?	Advance reservations, valet, plate cameras

Minute-by-minute plan

- **0–8:** Actors: driver, attendant, admin. Use cases: take ticket, park, pay, exit, view free spots.
- **8–18:** `ParkingLot` owns many `Level` owns many `ParkingSpot`; `Vehicle`, `Ticket`, `EntryGate`, `ExitGate`, `SpotAllocationStrategy`, `FeeStrategy`, `Payment`.
- **18–23:** Walk "car enters": gate → lot → strategy picks spot → spot claimed → ticket issued.
- **23–35:** Code `ParkingSpot.tryOccupy`, one allocation strategy, `FeeStrategy.fee(ticket, exitTime)`.

What a strong answer must include

- [ ] Size enums for vehicle and spot, with one fit rule.
- [ ] `Ticket` as its own class (spot, vehicle, entry time).
- [ ] Allocation and fee each behind an interface; fee separate from payment method.
- [ ] Atomic spot claim across gates.
- [ ] Free counts maintained in one place, not by scanning.

Likely follow-ups and how to answer

- **Two gates, last spot:** compare-and-set on the spot (or a per-size free queue); the loser retries, then shows "full". A lot-wide lock is correct but serialises every gate.
- **EV charging spots:** a new spot kind plus a charging fee class; no gate code changes.
- **Weekend rates / first hour free:** a new `FeeStrategy`, or a decorator around an existing one.
- **Lost ticket:** flat daily maximum; look up by plate if stored.

Where to study it

Ch 14 — 14.1 Design a Parking Lot

24.2 Drill — Elevator System

The prompt as the interviewer says it

> "Design the elevators for a building. People press buttons on floors and inside cars. How do you decide which elevator goes where?"

Clarifying questions to ask

Ask	Assume
Size?	3–4 cars, ~20 floors
Buttons?	Up/down per floor (hall calls), floor panel per car (car calls), door and emergency buttons
Capacity?	Weight limit; an overloaded car holds its doors open
Dispatch goal?	Short waits, few direction changes, swappable
Special modes?	Maintenance, fire mode (all cars to ground)
Out of scope?	Motor physics, access cards

Minute-by-minute plan

- **0–8:** Actors: passenger, operator. Use cases: call car, choose floor, open/close door, maintenance, fire mode.
- **8–18:** `ElevatorSystem` → many `ElevatorCar` (state, current floor, up-stops and down-stops as sorted sets), `Request` (floor, direction, kind), `Dispatcher` interface, `Door`.
- **18–23:** Walk "floor 7 presses up": dispatcher scores cars, adds a stop to one.
- **23–35:** Code the dispatcher's cost function and `ElevatorCar.step()` using the LOOK rule (keep going while stops lie ahead, then reverse).

What a strong answer must include

- [ ] Hall calls assigned to one car; car calls belong to their car.
- [ ] Sorted stop sets per direction, not a FIFO queue; you can say why FIFO zig-zags.
- [ ] Dispatch as Strategy (nearest, least busy, zoned).
- [ ] Car states with legal transitions; doors never open while moving.
- [ ] Button presses from many threads handled safely.

Likely follow-ups and how to answer

- **Why not nearest idle car?** A car already heading towards the caller in the same direction is often better. Cost = distance + direction-change penalty + load penalty.
- **Many presses at once:** one controller thread drains a request queue, so dispatch logic stays single-threaded.
- **Fire mode:** a system-wide mode that clears stops and sends every car down; cars consult it each step.
- **A car breaks mid-trip:** maintenance state, re-dispatch its hall calls to other cars.

Where to study it

Ch 14 — 14.5 Design an Elevator System

24.3 Drill — Library Management System

The prompt as the interviewer says it

> "Design a library system. Members borrow and return books, and late returns are fined."

Clarifying questions to ask

Ask	Assume
Multiple copies of a title?	Yes; each copy has a barcode and rack
Loan rules?	14 days, max 5 books, renew once
Reservations?	Per-title queue; next in line is notified on return
Fines?	Per day late; may differ by membership type
Search?	Title, author, subject, date
Out of scope?	E-books, inter-library loans, payment processing

Minute-by-minute plan

- **0–8:** Actors: member, librarian, system. Use cases: search, borrow, return, renew, reserve, pay fine.
- **8–18:** `Book` (title data) owns many `BookItem` (physical copy), `Member`, `Loan`, `Reservation`, `Fine`, `Catalog`, `FinePolicy`, `Notifier`.
- **18–23:** Walk "return a late book": close loan → compute fine → check reservation queue → notify.
- **23–35:** Code `borrow` (limit, availability, reservation honoured) and `returnItem`.

What a strong answer must include

- [ ] The **book vs copy** split; this is what the interviewer looks for first.
- [ ] `Loan` as a class with its own dates, not fields on `Member`.
- [ ] Borrow limit enforced in one place, per membership type.
- [ ] Fine rule behind an interface; notifications decoupled.
- [ ] Search separated from lending.

Likely follow-ups and how to answer

- **Two members, last copy:** the copy's available → loaned change is atomic; the loser is offered a reservation.
- **Reserved copy never collected:** hold N days, then a scheduled sweep passes it on.
- **Add DVDs and magazines:** generalise to a catalog item only where lending rules match; a non-lendable magazine must not inherit `borrow`.

Where to study it

Ch 15 — 15.1 Design a Library Management System

24.4 Drill — Amazon Locker Service

The prompt as the interviewer says it

> "Customers can have parcels delivered to a self-service locker. Design delivery, pickup, and what happens when nobody collects."

Clarifying questions to ask

Ask	Assume
Sites and sizes?	Many sites; lockers small/medium/large
Opening a locker?	One-time code sent to the customer
Pickup window?	3 days, then return to sender
Returns?	Customer drops an item with a code; courier collects
Site full?	Courier redirected to the nearest site with space
Out of scope?	Ordering, payments, hardware drivers

Minute-by-minute plan

- **0–8:** Actors: customer, delivery agent, ops admin. Use cases: assign locker, drop parcel, pick up, return item, expire.
- **8–18:** `LockerSite` owns many `Locker` (size, state), `Parcel`, `LockerAssignment` (parcel, locker, code, deadline), `LockerSelectionStrategy`, `CodeGenerator`, `Notifier`, `ExpiryJob`.
- **18–23:** Walk "pick up with code": find assignment by code → check deadline → open → free locker.
- **23–35:** Code `LockerSite.assign(parcel)` and `pickUp(code)`.

What a strong answer must include

- [ ] Ordered size enums; "smallest that fits" behind an interface.
- [ ] Locker states: free → reserved → occupied → free, plus out of service.
- [ ] Code on the assignment, single-use, with a deadline.
- [ ] Scheduled expiry that frees lockers and starts return-to-sender.
- [ ] Customer returns reuse the same assignment mechanism.

Likely follow-ups and how to answer

- **Two couriers, last medium locker:** reserve under a per-site lock or poll a per-size free queue; fall back to a larger size, then another site.
- **Repeated wrong codes:** count failures per keypad (a wrong code matches no assignment, so a per-assignment count cannot work) and lock the keypad for a few minutes after five in a row.
- **Jammed locker on arrival:** mark out of service and reassign; the code belongs to the assignment, so it survives.

Where to study it

Ch 14 — 14.7 Design a Shipping Locker System (Amazon Locker)

24.5 Drill — Vending Machine

The prompt as the interviewer says it

> "Design a vending machine. A user inserts money, picks a product, and gets the product and change."

This is a State-pattern question; reach the states quickly.

Clarifying questions to ask

Ask	Assume
Payment?	Coins and notes; cards later
Order?	Pay first, then pick
Cancel?	Full refund of what was inserted
No change available?	Refuse and refund before dispensing
Products?	Slots (A1, B3) with product and count
Out of scope?	Telemetry, cooling

Minute-by-minute plan

- **0–8:** Actors: customer, operator. Use cases: insert, select, cancel, dispense, restock.
- **8–18:** `VendingMachine` (context) → `VendingState` with `Idle`, `HasMoney`, `Dispensing`, `SoldOut`, `Service`; `Inventory`, `Slot`, `Coin`, `ChangeMaker`.
- **18–23:** Walk the state diagram, including cancel and sold-out.
- **23–35:** Code the state interface, two states, and `ChangeMaker.makeChange`.

What a strong answer must include

- [ ] Each state lists the events it accepts; illegal events fail clearly.
- [ ] State pattern justified: each mode's rules in one class, no growing `switch`.
- [ ] Money as integer minor units, never `double`.
- [ ] Change checked against real coin stock **before** dispensing.
- [ ] Restock as a state, so sales stop during it.

Likely follow-ups and how to answer

- **Greedy change?** Works for normal coin sets, fails for odd ones or depleted stock; a small search is the fallback behind the same interface.
- **Card payment:** a payment-method abstraction; card authorises the exact price, so no change.
- **Product jams:** back to `HasMoney`, offer refund or another pick.
- **Enum vs State classes:** an enum with a switch is fine for four stateless modes; classes pay off once states carry data or multiply.

Where to study it

Ch 14 — 14.2 Design a Vending Machine

24.6 Drill — Online Blackjack Game

The prompt as the interviewer says it

> "Design an online Blackjack game: several players at a table against a dealer, with betting."

Clarifying questions to ask

Ask	Assume
Players per table?	1–5 plus the dealer
Decks?	Six-deck shoe, reshuffled at a cut card
Actions?	Hit, stand, double; split later
Dealer rule?	Hits below 17, stands on all 17s
Payouts?	1:1 win, 3:2 blackjack, push returns bet
Out of scope?	Side bets, insurance, networking details

Minute-by-minute plan

- **0–8:** Actors: player, dealer (system). Use cases: join, bet, deal, hit, stand, double, settle.
- **8–18:** `Card` (`Rank`, `Suit`), `Deck`, `Shoe`, `Hand`, `Player`, `Dealer`, `Table`, `Round`, `Bet`, `DealerPolicy`.
- **18–23:** Walk one round: bets → deal → players act in turn → dealer plays → settle.
- **23–35:** Code `Hand.bestValue()` and `Round.settle()`.

What a strong answer must include

- [ ] A game-agnostic `Card`/`Deck`, reusable for other card games.
- [ ] The ace rule inside `Hand` only.
- [ ] `Round` with phases, separate from the long-lived `Table`.
- [ ] Dealer rule as a policy, so "hit soft 17" is a new class.
- [ ] Bets debited once at start, settled once; an injectable seeded shuffle for tests.

Likely follow-ups and how to answer

- **Ace value:** count aces as 1; if total + 10 ≤ 21, add 10 once. That also yields the "soft" flag.
- **Splitting:** a player holds a list of `Hand`s, each with its own `Bet`; design for the list from the start.
- **Online multiplayer:** each table processes player commands from a queue on one thread, so game state needs no locks.
- **Reuse for Poker:** deck, card and shoe carry over; hand evaluation and round phases do not.

Where to study it

Ch 17 — Design Blackjack and a Deck of Cards

24.7 Drill — Ride Sharing (practice session)

The prompt as the interviewer says it

> "Design a ride-sharing service like Uber. Riders request rides, drivers accept them, and the rider pays at the end."

Clarifying questions to ask

Ask	Assume
Matching rule?	Nearest available driver within a radius; swappable
Ride types?	Standard and premium; carpool later
Fare?	Base + distance + time, with surge multiplier
Cancellation?	Either side may cancel; a fee after driver assignment
Ratings?	Both sides rate after the trip
Out of scope?	Maps/routing, real payment gateway

Minute-by-minute plan

- **0–8:** Actors: rider, driver, payment system. Use cases: request, match, accept, start, complete, pay, rate, cancel.
- **8–18:** `Rider`, `Driver` (status, `Vehicle`, `Location`), `RideRequest`, `Trip` (state), `MatchingStrategy`, `FareStrategy`, `DriverLocator`, `TripService`, `Payment`.
- **18–23:** Walk "request to completion": request → match → offer → accept → start → complete → fare → pay.
- **23–35:** Code `TripService.request` with matching and atomic driver claim, plus `Trip` state transitions.

What a strong answer must include

- [ ] `Trip` state machine: requested → driver assigned → in progress → completed, with cancelled from the early states.
- [ ] Matching and fare each as Strategy (nearest/best-rated; normal/surge).
- [ ] Driver availability claimed atomically.
- [ ] Location lookup behind its own class (a grid or geohash index), not a scan.
- [ ] Observer for live status updates to the rider.

Likely follow-ups and how to answer

- **Two requests match one driver:** compare-and-set the driver's status from available to assigned; the loser is re-matched to the next candidate.
- **Driver declines or times out:** offer to the next candidate; cap attempts, then tell the rider.
- **Carpool:** a trip holds several riders and seats; matching checks detour cost. A new strategy plus a seats field, not a new service.
- **Surge:** a fare decorator using a demand/supply ratio per area.

Where to study it

Ch 20 — 20.4 Design a Ride Sharing Service (Uber)

24.8 Drill — Meeting Scheduler

The prompt as the interviewer says it

> "Design a meeting scheduler for an office. People book rooms for meetings and invite colleagues."

Clarifying questions to ask

Ask	Assume
Room choice?	System picks a free room with enough capacity; user may name one
Time model?	Start and end on one day; half-open intervals, so 10–11 and 11–12 do not clash
Invitations?	Invitees accept or decline; organiser sees replies
Notifications?	On invite, change and cancel
Recurring meetings?	Later extension
Out of scope?	External calendars, time zones across offices

Minute-by-minute plan

- **0–8:** Actors: organiser, invitee, admin. Use cases: book, invite, respond, cancel, view room or person calendar.
- **8–18:** `Scheduler`, `Room` (capacity, own `Calendar` of bookings), `Meeting` (interval, organiser, room), `Invitation` (status), `Interval`, `RoomSelectionStrategy`, `Notifier`.
- **18–23:** Walk "book": filter rooms by capacity → strategy orders them → first room with no overlap is booked → invitations sent.
- **23–35:** Code `Interval.overlaps`, `Room.tryReserve` (sorted bookings, one `lowerEntry(end)` check) and `Scheduler.book`.

What a strong answer must include

- [ ] An `Interval` value type with a correct half-open overlap test.
- [ ] Bookings per room in a sorted map; clash check in O(log n).
- [ ] Check and insert under one per-room lock.
- [ ] Room selection as Strategy (smallest fitting, nearest).
- [ ] Invitations with RSVP status; notifications through Observer.

Likely follow-ups and how to answer

- **Two users, same room, overlapping times:** lock per room; when trying rooms in turn, release one lock before taking the next, so no deadlock.
- **First slot when all attendees are free:** gather busy intervals, sort, merge overlaps, scan the gaps for the required length.
- **Recurring meetings:** expand into instances; fail the whole booking on any clash, or offer to skip clashing dates.
- **Room goes out of service:** cancel or move its future meetings and notify organisers.

Where to study it

Ch 16 — Design a Meeting Scheduler

24.9 Drill — Movie Ticket Booking System

The prompt as the interviewer says it

> "Design BookMyShow: users find a movie, pick a show and seats, and pay."

Clarifying questions to ask

Ask	Assume
Scope?	Many cities, cinemas, screens and shows
Seat selection?	User picks exact seats on a seat map
Seat types and pricing?	Regular/premium; price per show and seat type
Hold time?	Seats held 10 minutes while the user pays
Cancellation?	Allowed up to a cutoff, with refund rules
Out of scope?	Reviews, food ordering, real payment gateway

Minute-by-minute plan

- **0–8:** Actors: customer, cinema admin, payment system. Use cases: search, view seats, hold, pay, cancel, add show.
- **8–18:** `Cinema` owns `Screen` owns `Seat`; `Movie`; `Show` (screen, movie, time); `ShowSeat` (seat state per show); `Booking`; `Payment`; `PricingStrategy`.
- **18–23:** Walk "book two seats": hold seats → create pending booking → pay → confirm, or release on timeout.
- **23–35:** Code `ShowSeat` hold/confirm/release and `BookingService.hold(show, seats, user)` as all-or-nothing.

What a strong answer must include

- [ ] `Seat` (physical) vs `ShowSeat` (state for one show); this is the key split.
- [ ] Seat states: available → held → booked, with hold expiry back to available.
- [ ] Holding several seats is all-or-nothing.
- [ ] Pricing as Strategy; booking confirmation via Observer.
- [ ] Payment failure releases the hold.

Likely follow-ups and how to answer

- **Two users click the same seat:** lock the show's seats in a fixed order (by seat id) and check-and-hold together; the second user sees "taken". Fixed order prevents deadlock between overlapping selections.
- **Payment succeeds after the hold expired:** check hold ownership at confirm; if lost, refund automatically.
- **Show cancelled by cinema:** cancel all bookings for the show and refund, notifying each customer.
- **Many servers:** the hold must live in shared storage with a conditional update, not in one JVM's memory.

Where to study it

Ch 15 — 15.2 Design a Movie Ticket Booking System

24.10 Drill — Car Rental System

The prompt as the interviewer says it

> "Design a car rental system. Customers reserve a car at a branch for some dates and return it later."

Clarifying questions to ask

Ask	Assume
Reserve a model or a specific car?	A vehicle type; the car is assigned at pickup
Branches?	Several; one-way rentals allowed
Pricing?	Daily rate per type, plus add-ons (insurance, GPS)
Late return / damage?	Extra charges at return
Cancellation?	Free until 24 hours before pickup
Out of scope?	Fleet maintenance scheduling, loyalty

Minute-by-minute plan

- **0–8:** Actors: customer, branch agent, admin. Use cases: search, reserve, pick up, return, cancel, pay.
- **8–18:** `Branch` holds many `Vehicle` (type, status); `Customer`; `Reservation` (type, dates, pickup and drop branch, state); `RentalAgreement`; `PricingPolicy`; `Invoice`; `AddOn`.
- **18–23:** Walk "return late": close rental → price base + add-ons + late fee → invoice → vehicle available at drop branch.
- **23–35:** Code availability for a date range and `Reservation` state transitions.

What a strong answer must include

- [ ] Availability as interval overlap, not a single "is free" flag.
- [ ] Reservation lifecycle: reserved → picked up → returned, plus cancelled and no-show.
- [ ] Pricing as Strategy; add-ons composable (Decorator or a list).
- [ ] One-way rental moves the vehicle to the drop branch.
- [ ] Atomic reserve against overlapping requests for the last car of a type.

Likely follow-ups and how to answer

- **Last SUV, two customers:** per-branch, per-type lock (or a count with compare-and-set) around check and reserve.
- **Overbooking:** allow a small configured margin per type; upgrade when short.
- **Dynamic pricing:** a new pricing class reading demand; nothing else changes.
- **Car breaks during rental:** swap vehicle on the same agreement; the agreement, not the car, carries the money.

Where to study it

Ch 16 — Design a Car Rental System

24.11 Drill — ATM

The prompt as the interviewer says it

> "Design an ATM. A customer inserts a card, enters a PIN, and withdraws cash or checks the balance."

Clarifying questions to ask

Ask	Assume
Transactions?	Withdraw, deposit, balance, transfer between own accounts
Who holds balances?	The bank, via a remote service
PIN rules?	Three wrong tries keep the card
Cash?	Cassettes per denomination; limited stock
Limits?	Daily withdrawal limit per card
Out of scope?	Bank's internal ledger, hardware drivers

Minute-by-minute plan

- **0–8:** Actors: customer, bank system, technician. Use cases: authenticate, withdraw, deposit, balance, refill cash.
- **8–18:** `ATM` (context) with states `Idle`, `CardInserted`, `Authenticated`, `Dispensing`; `Card`; `BankService` interface; `CashDispenser` with a chain of denomination handlers; `Transaction` subclasses.
- **18–23:** Walk "withdraw": check cash → debit via bank → dispense → print receipt; reverse on dispense failure.
- **23–35:** Code the state transitions and the dispenser chain.

What a strong answer must include

- [ ] ATM states with events allowed in each.
- [ ] Bank behind an interface; ATM never holds balances.
- [ ] Note dispensing as Chain of Responsibility (largest denomination first).
- [ ] Order: check cash → debit → dispense → on failure, reverse the debit.
- [ ] PIN retry limit and card retention.

Likely follow-ups and how to answer

- **Debited but no cash:** send a reversal with the same transaction id; the bank treats it idempotently.
- **Two ATMs, one account:** the bank serialises debits per account; the ATM just handles a "declined" reply.
- **Network drop mid-transaction:** retry with the same id; idempotency stops a double debit.
- **Add cardless (QR) withdrawal:** a new authentication state; the rest of the flow is unchanged.

Where to study it

Ch 14 — 14.4 Design an ATM

24.12 Drill — Chess Game

The prompt as the interviewer says it

> "Design a two-player chess game."

Clarifying questions to ask

Ask	Assume
Players?	Two humans; computer player later
Full rules?	All moves, including castling, en passant, promotion
End conditions?	Checkmate, stalemate, resignation, draw by agreement
Clocks?	Optional time control
Undo?	Yes, for practice games
Out of scope?	Ratings, online matchmaking

Minute-by-minute plan

- **0–8:** Actors: two players. Use cases: start, move, undo, resign, offer draw.
- **8–18:** `Game` (status, turn, history), `Board` owns 64 `Square`s, abstract `Piece` with subclasses, `Move` (from, to, captured, special kind), `Player`, `MoveValidator`.
- **18–23:** Walk "make a move": piece's pseudo-legal moves → apply to a copy → reject if own king in check → commit → update status.
- **23–35:** Code `Piece.candidateMoves` for two pieces and `Game.makeMove` with the check test.

What a strong answer must include

- [ ] Piece movement through polymorphism, one subclass each; sliding pieces share a helper.
- [ ] `Move` as an object (Command) storing enough to undo.
- [ ] Legality = pseudo-legal + "own king not in check after the move".
- [ ] Game status computed after every move (check, mate, stalemate).
- [ ] Turn enforcement in `Game`, not in pieces.

Likely follow-ups and how to answer

- **Detect checkmate:** in check and no legal move exists; generate all moves and test each.
- **Castling rules:** needs "has moved" flags, empty path, and no attacked squares on the king's path.
- **Add an AI player:** a `Player` strategy that picks a move; the game loop does not change.
- **Why not a giant switch on piece type?** Adding a fairy piece would edit it; a subclass is closed for modification.

Where to study it

Ch 17 — Design Chess

24.13 Drill — Hotel Management System

The prompt as the interviewer says it

> "Design a hotel management system: booking rooms, check-in, check-out, and billing."

Clarifying questions to ask

Ask	Assume
Book a room type or a room?	A type; the room is assigned at check-in
Date model?	Nights as half-open date ranges
Extra services?	Room service, laundry, added to the bill
Cancellation and no-show?	Free until a cutoff; one night charged for no-show
Housekeeping?	Room must be cleaned before the next guest
Out of scope?	Channel managers, loyalty

Minute-by-minute plan

- **0–8:** Actors: guest, receptionist, housekeeper. Use cases: search, reserve, check in, add charge, check out, cancel.
- **8–18:** `Hotel` owns `Room` (type, status); `Guest`; `Reservation` (type, date range, state); `Stay`; `Invoice` with `Charge` lines; `RoomPricing`; `HousekeepingTask`.
- **18–23:** Walk "check out": total charges → pay → room marked dirty → housekeeping task created.
- **23–35:** Code availability per room type over a date range and the reservation state machine.

What a strong answer must include

- [ ] Availability = count of rooms of a type minus overlapping reservations, per night.
- [ ] Reservation states: booked → checked in → checked out, plus cancelled and no-show.
- [ ] Room status separate from reservation status (occupied, dirty, clean, out of order).
- [ ] Pricing as Strategy (season, weekend).
- [ ] Charges as line items, so new services need no invoice change.

Likely follow-ups and how to answer

- **Last room, two bookings:** lock per room type and date range check-and-insert; or a per-night counter decremented atomically.
- **Early check-in / late check-out:** a charge line plus a room-status check.
- **Overbooking:** allow a configured margin and a "walk" procedure to a partner hotel.
- **Group bookings:** a parent reservation holding several room reservations, booked all-or-nothing.

Where to study it

Ch 15 — 15.4 Design a Hotel Management System

24.14 Drill — Food Delivery (practice session)

The prompt as the interviewer says it

> "Design a food delivery app like Swiggy: customers order from restaurants, and delivery agents bring the food."

Clarifying questions to ask

Ask	Assume
One restaurant per order?	Yes; the cart resets if the user switches restaurant
Restaurant capacity?	A kitchen cooks at most N items at once
Agent assignment?	Nearest free agent when the food is ready; swappable
Tracking?	Customer sees order status changes
Cancellation?	Until cooking starts; prepaid orders are refunded
Out of scope?	Routing maps, payment gateway internals

Minute-by-minute plan

- **0–8:** Actors: customer, restaurant, delivery agent, admin. Use cases: browse, add to cart, place order, accept, prepare, assign agent, deliver, cancel.
- **8–18:** `Restaurant` (menu, capacity, rating), `MenuItem`, `Cart`, `Order` (state), `DeliveryAgent` (status, location), `RestaurantSelectionStrategy`, `AgentAssignmentStrategy`, `OrderService`, `Notifier`.
- **18–23:** Walk "place order to delivered": cart → order placed → restaurant accepts (capacity taken) → ready → agent assigned → picked up → delivered (capacity freed).
- **23–35:** Code `Order` state transitions and capacity handling in `OrderService`.

What a strong answer must include

- [ ] Order state machine with who may trigger each transition.
- [ ] Kitchen capacity taken on accept, released when the food is ready or the order is cancelled; a refusal for a full kitchen refunds a prepaid order.
- [ ] Restaurant selection and agent assignment as separate Strategies.
- [ ] Atomic claim of a free agent.
- [ ] Status updates through Observer.

Likely follow-ups and how to answer

- **Two orders, one agent:** compare-and-set the agent's status; the loser goes to the next candidate.
- **Restaurant never accepts:** a timeout auto-cancels and refunds.
- **Batch two orders to one agent:** assignment strategy that allows a second pickup within a detour limit.
- **Pick the cheapest restaurant for a whole cart (FoodKart):** a selection strategy across restaurants serving every item.

Where to study it

Ch 19 — Design an Online Food Delivery Service (Swiggy) + FoodKart

24.15 Drill — Amazon Online Shopping System

The prompt as the interviewer says it

> "Design the core of Amazon: browse products, add to cart, check out, and track the order."

Clarifying questions to ask

Ask	Assume
Sellers?	Many; a product may be sold by several
Stock?	Per product per warehouse
Reserve stock when?	At checkout, released if payment fails
Payments?	Card, wallet, cash on delivery
Order changes?	Cancel before shipping; return after delivery
Out of scope?	Recommendations, search ranking

Minute-by-minute plan

- **0–8:** Actors: customer, seller, admin, payment system. Use cases: search, add to cart, checkout, pay, track, cancel, return, review.
- **8–18:** `Product`, `Catalog`, `Cart` with `CartItem`, `Order` with `OrderItem` (price copied), `Inventory`, `Payment` (Strategy), `Shipment`, `Customer`, `Notifier`.
- **18–23:** Walk "checkout": validate cart → reserve stock → create order → pay → confirm, or release on failure.
- **23–35:** Code `CheckoutService.checkout` and `Order` state transitions.

What a strong answer must include

- [ ] `OrderItem` copies price at purchase time; the cart reads live prices.
- [ ] Stock reserved atomically for all lines, released on failure.
- [ ] Order states: placed → paid → shipped → delivered, plus cancelled and returned.
- [ ] Payment and shipping methods as Strategy.
- [ ] Notifications via Observer on state change.

Likely follow-ups and how to answer

- **Price changes while in the cart:** recheck at checkout and show the new price before paying.
- **Last unit, two buyers:** conditional decrement on stock; the loser gets "out of stock" at checkout.
- **Partial cancellation:** cancel per `OrderItem`, refund that line, release its stock.
- **Payment timeout:** reservation has an expiry; a sweeper releases stock.

Where to study it

Ch 19 — Design Amazon — Online Shopping System

24.16 Drill — Stack Overflow

The prompt as the interviewer says it

> "Design Stack Overflow: users ask questions, answer them, vote, and earn reputation."

Clarifying questions to ask

Ask	Assume
Content types?	Question, answer, comment; tags on questions
Voting?	Up/down on questions and answers, one vote per user per post
Reputation?	Changes on votes and accepted answers; unlocks privileges
Bounties and badges?	Yes, as extensions
Moderation?	Close, delete, flag
Out of scope?	Full-text search engine internals, chat

Minute-by-minute plan

- **0–8:** Actors: guest, member, moderator, system. Use cases: ask, answer, comment, vote, accept, search, close.
- **8–18:** `User` (reputation), abstract `Post` → `Question`, `Answer`; `Comment`; `Vote`; `Tag`; `Badge`; `Bounty`; `ReputationService`; `SearchService`.
- **18–23:** Walk "upvote an answer": check privilege → record vote (unique per user and post) → update score → reputation event to the author.
- **23–35:** Code `VoteService.vote` and the reputation listener.

What a strong answer must include

- [ ] Shared `Post` base for voting and commenting; `Question` adds title, tags and accepted answer.
- [ ] `Vote` keyed by (user, post) so a second vote replaces, never adds.
- [ ] Reputation changed by events (Observer), not inline in the vote code.
- [ ] Privileges as reputation thresholds in one place.
- [ ] Search separate from posting.

Likely follow-ups and how to answer

- **Concurrent votes on a hot post:** atomic score counter; the unique (user, post) key stops double voting.
- **Changing a vote:** reverse the old reputation delta, then apply the new one.
- **Bounties:** reputation is escrowed when offered and paid on award or expiry.
- **Self-voting / vote rings:** a rule in the vote service; detection can run later as a job.

Where to study it

Ch 18 — 18.1 Design Stack Overflow

24.17 Drill — Restaurant Management System

The prompt as the interviewer says it

> "Design a restaurant system: table reservations, taking orders, the kitchen, and the bill."

Clarifying questions to ask

Ask	Assume
Reservations and walk-ins?	Both; walk-ins join a waitlist
Tables?	Sizes 2/4/6; tables may be joined
Ordering?	Waiter enters orders per table; items go to kitchen stations
Bill?	Per table, with tax and tip; may be split
Menu changes?	Items can be marked unavailable
Out of scope?	Inventory purchasing, payroll

Minute-by-minute plan

- **0–8:** Actors: guest, host, waiter, chef, cashier. Use cases: reserve, seat, order, prepare, serve, bill, pay.
- **8–18:** `Table` (size, status), `Reservation`, `WaitlistEntry`, `Menu` with `MenuItem`, `Order` with `OrderItem` (state), `Kitchen` with stations, `Bill`, `TableAllocator`.
- **18–23:** Walk "order to bill": waiter places order → items sent to stations → ready → served → bill.
- **23–35:** Code `TableAllocator` (smallest table that fits) and `OrderItem` state transitions.

What a strong answer must include

- [ ] Table allocation as Strategy, with waitlist fallback.
- [ ] Orders sent to the kitchen as Command objects (queued, can be cancelled).
- [ ] `OrderItem` state: ordered → cooking → ready → served.
- [ ] "Table ready" and "dish ready" via Observer.
- [ ] Bill built from order items plus tax/tip policy; split as a separate step.

Likely follow-ups and how to answer

- **Reservation vs walk-in for the same table:** hold reserved tables from a time before the slot; walk-ins only get tables free for a full sitting.
- **Split the bill:** by items or evenly; a split strategy producing several bills.
- **Item runs out mid-service:** mark unavailable; pending orders with it are flagged to the waiter.
- **Several waiters ordering for one table:** lock the order per table, or append-only items.

Where to study it

Ch 16 — Design a Restaurant Management System

24.18 Drill — Facebook

The prompt as the interviewer says it

> "Design the core of Facebook: profiles, friends, posts, a news feed and privacy."

Clarifying questions to ask

Ask	Assume
Friends or followers?	Both: mutual friendship and one-way follow
Post visibility?	Public, friends, only me, custom list
Feed?	Recent posts from friends, followed pages and groups, ranked
Interactions?	Like, comment, share
Groups and pages?	Yes, simple versions
Out of scope?	Ads, messaging, media storage

Minute-by-minute plan

- **0–8:** Actors: user, page admin, system. Use cases: send/accept friend request, post, comment, like, view feed, search.
- **8–18:** `User`, `Profile`, `FriendRequest` (state), `Friendship`, `Follow`, `Post` (audience), `Comment`, `Reaction`, `Group`, `Page`, `VisibilityPolicy`, `FeedService` with `FeedRanker`.
- **18–23:** Walk "load feed": collect candidate posts → filter by visibility → rank → page.
- **23–35:** Code `VisibilityPolicy.canSee(viewer, post)` and the feed assembly.

What a strong answer must include

- [ ] Friendship (two-sided, via request) distinct from follow (one-sided).
- [ ] One visibility policy reused by feed, profile and search.
- [ ] Feed ranking as Strategy.
- [ ] A stated choice of pull (merge at read) vs push (fan-out at write), with the trade-off.
- [ ] Friend request state: pending → accepted / rejected.

Likely follow-ups and how to answer

- **Push or pull?** Pull is simpler and right for an in-memory design; push makes reads fast but costs writes for users with many friends. Hybrid: push for normal users, pull for celebrities.
- **Privacy changed after posting:** check visibility at read time, never cache it into the feed.
- **Friend suggestions:** friends-of-friends counted by mutual friends (two-hop search).
- **Unfriend:** remove both sides of the friendship atomically.

Where to study it

Ch 18 — 18.4 Design Facebook

24.19 Drill — Online Stock Brokerage System

The prompt as the interviewer says it

> "Design an online stock brokerage: users fund an account, place buy and sell orders, and see their portfolio."

Clarifying questions to ask

Ask	Assume
Order types?	Market and limit; stop-loss later
Who matches orders?	An external exchange; we route and track
Partial fills?	Yes
Funds?	Blocked when a buy is placed, released on cancel
Alerts?	Price alerts and watchlists
Out of scope?	Margin trading, tax reports

Minute-by-minute plan

- **0–8:** Actors: investor, exchange, admin. Use cases: deposit, place order, cancel, receive fill, view portfolio, set alert.
- **8–18:** `Account` (cash, blocked cash), `Portfolio` with `Position`s, abstract `Order` → `MarketOrder`, `LimitOrder` (state), `Fill`, `ExchangeGateway` (Adapter), `OrderService`, `Watchlist`, `PriceAlert`.
- **18–23:** Walk "buy with partial fills": validate → block funds → send → fills update position and cash → filled or cancelled remainder releases funds.
- **23–35:** Code `OrderService.place` and `onFill`.

What a strong answer must include

- [ ] Funds or shares blocked at placement, not at fill.
- [ ] Order states: new → open → partially filled → filled / cancelled / rejected.
- [ ] Exchange behind an Adapter interface.
- [ ] Money as `BigDecimal` or integer units.
- [ ] Price alerts through Observer.

Likely follow-ups and how to answer

- **Two orders spend the same cash:** lock per account around check-and-block.
- **Fill arrives after cancel:** the exchange is the truth; apply the fill, cancel only the remainder.
- **Duplicate fill messages:** keep processed fill ids; ignore repeats (idempotency).
- **How would the exchange match?** Per symbol, a price-sorted book per side, matched by price then time.

Where to study it

Ch 19 — Design an Online Stock Brokerage System

24.20 Drill — Jigsaw Puzzle

The prompt as the interviewer says it

> "Design a jigsaw puzzle: pieces, the board, how to tell whether two pieces fit, and an algorithm to solve it."

Clarifying questions to ask

Ask	Assume
Piece shape?	Rectangular grid; each piece has four sides
Side kinds?	Flat (border), tab (sticks out), blank (goes in)
How do two sides fit?	A tab fits a blank with the same shape id
Rotation?	Pieces may be rotated
Solve or just play?	Both: a user places pieces; a solver can finish
Out of scope?	Image processing, 3-D puzzles

Minute-by-minute plan

- **0–8:** Actor: player (the solver is part of the system). Use cases: start (includes validating the box), place piece (includes choosing a rotation), remove, auto-solve.
- **8–18:** `Puzzle`, `Board` (grid of `Placement(piece, rotation)`), immutable `Piece` (four `Side`s), `Side` (kind, shape id), `Direction` enum (TOP, RIGHT, BOTTOM, LEFT), `SolverStrategy`.
- **18–23:** Walk the solver: corners (two flat sides) → borders (one flat side) → inside, each by matching neighbours.
- **23–35:** Code `Side.fits(other)`, `Piece.side(direction, rotation)`, `Board.check` and the solver loop with a side index.

What a strong answer must include

- [ ] Sides as their own type with a fit rule, not booleans on the piece.
- [ ] Rotation stored in the placement, not the piece; the side facing `d` after `r` turns is `sides[(d − r) mod 4]`.
- [ ] Piece category from flat-side count: corner, border, interior.
- [ ] Solver that fills cells in order and looks up candidates by shape id (hash map), not by scanning all pieces.
- [ ] Board validates a placement against already-placed neighbours.

Likely follow-ups and how to answer

- **Complexity:** with unique shapes and an index from side to pieces, each cell is O(1) average, so O(n) total; scanning every piece is $O(n^2)$. Look-alike pieces force backtracking, so budget the search.
- **Two pieces fit the same spot:** add a picture-similarity score as a tie-breaker, or backtrack.
- **Undo a placement:** placements as Command objects on a stack.
- **Non-rectangular puzzles:** sides become a list and `Edge` a direction count; most code is unchanged.

Where to study it

Ch 17 — Design a Jigsaw Puzzle

24.21 Drill — Airline Management System

The prompt as the interviewer says it

> "Design an airline reservation system: search flights, book seats, check in."

Clarifying questions to ask

Ask	Assume
Route vs dated departure?	A flight number recurs; each date is its own instance
Multi-leg trips?	Yes, booked all-or-nothing
Seat classes?	Economy, business; seat selection optional
Fares?	Per class, varies with time to departure
Changes?	Cancel with refund rules; crew assignment for admin
Out of scope?	Loyalty, baggage tracking

Minute-by-minute plan

- **0–8:** Actors: customer, agent, admin, crew. Use cases: search, book, pay, select seat, check in, cancel, assign crew.
- **8–18:** `Airport`, `Flight` (route, number), `FlightInstance` (date, aircraft, status), `Aircraft` with `Seat`s, `FlightSeat` (per instance), `Itinerary` with `Reservation`s, `Passenger`, `FarePolicy`, `Payment`.
- **18–23:** Walk "book a two-leg itinerary": hold seats on both instances → pay → confirm both, or release both.
- **23–35:** Code the itinerary hold as all-or-nothing and `FlightSeat` states.

What a strong answer must include

- [ ] `Flight` vs `FlightInstance` split, and seat state per instance.
- [ ] All-or-nothing booking across legs with release on failure.
- [ ] Fare rules as Strategy.
- [ ] Booking states: held → confirmed → checked in / cancelled.
- [ ] Flight status changes notify passengers (Observer).

Likely follow-ups and how to answer

- **Two people, same seat:** lock seats in a fixed order across instances to avoid deadlock; check-and-hold together.
- **Flight cancelled:** rebook passengers on the next instance or refund.
- **Overbooking:** allow a margin per class; deny boarding by a priority rule at check-in.
- **Multi-airline itineraries:** an adapter per partner airline behind one booking interface.

Where to study it

Ch 16 — Design an Airline Management System

24.22 Drill — ESPNcricinfo

The prompt as the interviewer says it

> "Design a cricket site like ESPNcricinfo: live scores, scorecards, commentary and player stats."

Clarifying questions to ask

Ask	Assume
Formats?	Test, ODI, T20
Data entry?	An admin enters each ball
Live view?	Scorecard and commentary update per ball
Stats?	Career stats per player, per format
Tournaments?	Series and tournaments with points tables
Out of scope?	Video, fantasy games

Minute-by-minute plan

- **0–8:** Actors: viewer, scorer/admin, system. Use cases: record ball, view scorecard, follow match, view stats, manage tournament.
- **8–18:** `Tournament`, `Match` (format), `Team`, `Player`, `Innings`, `Over`, `Ball` (runs, extras, wicket), `Wicket`, `Commentary`, `Scorecard`, `MatchFormatRules`.
- **18–23:** Walk "record a ball": validate → append ball → update innings totals and strike → notify subscribers.
- **23–35:** Code `Innings.record(ball)` including extras and strike rotation.

What a strong answer must include

- [ ] `Ball` as the source event; totals and stats derived from balls.
- [ ] Extras handled correctly: wide and no-ball do not count as legal deliveries.
- [ ] Strike rotation on odd runs and at the end of an over.
- [ ] Format rules (overs, powerplays) behind an interface or Template Method.
- [ ] Live updates via Observer.

Likely follow-ups and how to answer

- **Scorer corrects a ball:** replace that event and recompute derived totals.
- **Career stats fast to read:** maintain aggregates updated from ball events, rebuildable from the log.
- **Many viewers:** one writer per match; readers get immutable snapshots.
- **Rain-reduced match:** a format rule computes the revised target; the ball model is unchanged.

Where to study it

Ch 18 — 18.2 Design Cricinfo

24.23 Drill — LinkedIn

The prompt as the interviewer says it

> "Design LinkedIn: profiles, connections, a feed, jobs and messages."

Clarifying questions to ask

Ask	Assume
Connections?	Two-sided, by invitation; show 1st/2nd/3rd degree
Profile?	Experience, education, skills, endorsements
Jobs?	Companies post; members apply
Feed?	Posts from connections and followed companies
Messaging?	One-to-one, between connections
Out of scope?	Ads, premium analytics

Minute-by-minute plan

- **0–8:** Actors: member, recruiter, company admin. Use cases: edit profile, connect, accept, post, view feed, post job, apply, message.
- **8–18:** `Member`, `Profile` (`Experience`, `Education`, `Skill`), `Invitation` (state), `Connection`, `Company`, `Job`, `Application` (state), `Post`, `Message`, `FeedService`, `ConnectionGraph`.
- **18–23:** Walk "accept invitation": invitation → accepted → connection stored for both sides → feed and suggestions update.
- **23–35:** Code `ConnectionGraph.degree(a, b)` with a bounded breadth-first search, and `Invitation` transitions.

What a strong answer must include

- [ ] Invitation states: pending → accepted / ignored / withdrawn.
- [ ] Connection symmetric and created for both members in one step.
- [ ] Degree by BFS limited to depth 3.
- [ ] Job applications with their own state (applied → reviewed → rejected / offered).
- [ ] Feed ranking as Strategy; notifications via Observer.

Likely follow-ups and how to answer

- **Degree on a huge graph:** bidirectional BFS from both ends, stop at depth 3.
- **"People you may know":** second-degree members ranked by mutual connections.
- **Messages to non-connections:** a permission check (premium or recruiter) before sending.
- **Endorsements:** a (skill, endorser) pair stored once per endorser.

Where to study it

Ch 18 — 18.3 Design LinkedIn

Chapter summary

- The patterns repeat across problems: an atomic claim of a scarce resource (spot, seat, room, driver, locker), a state machine for the main entity, a Strategy for the rule most likely to change, and Observer for notifications. Recognise which one the interviewer is probing.